

Analysis of Algorithms Laboratory

Lab 08: In-place Swap, Counting Sort Variants, and Closed Hashing

Aispur P. Santiago

School of Mathematical and Computational Sciences

Yachay Tech University

San Miguel de Urcuquí - Imbabura, Ecuador

April 13, 2026

1 Introduction

This laboratory assignment consists of four main components:

1. Determine whether it is possible to exchange numeric values of two variables (e.g., u and v) without using any extra storage, and explain how.
2. Implement and analyze the *ComparisonCountingSort* algorithm, including testing methodology.
3. Implement and analyze the *DistributionCountingSort* algorithm, test it, and compare it with *ComparisonCountingSort* in terms of complexity and implementation differences.
4. Implement and analyze a hashing algorithm using C++ constraints (no dictionaries or libraries), using *closed hashing* as the collision resolution method.

2 Problem 1: Swapping Two Variables Without Extra Storage

2.1 Algorithm Description

Yes, it is possible to exchange the numeric values of two variables without using any extra storage (i.e., without a temporary third variable). Two classical methods are commonly discussed in algorithm courses:

1. **Arithmetic-based swap (addition/subtraction).**
2. **Bitwise XOR swap (for integers).**

Both methods use only constant extra space, i.e., $\Theta(1)$ additional storage, satisfying the “no extra storage” requirement.

2.2 Correctness and Practical Remarks

- Both approaches are correct because they preserve the information in (u, v) through reversible operations.
- The arithmetic method can overflow when using bounded integer types.
- XOR swap avoids arithmetic overflow but is limited to integer representations and is mainly of theoretical interest; in practice, readability and compiler optimizations typically favor standard swaps.

2.3 Complexity Analysis

Both swap methods perform a constant number of primitive operations:

$$T(n) = \Theta(1), \quad S(n) = \Theta(1).$$

3 Problem 2: ComparisonCountingSort

3.1 Algorithm Description

ComparisonCountingSort is a sorting technique based on pairwise comparisons. For each element $A[i]$, the algorithm counts how many elements are strictly smaller than it; that count determines the final position of $A[i]$ in the sorted output.

A crucial implementation detail is **handling duplicates**. If equal keys exist, naive counting can assign identical positions to multiple elements. To ensure unique positions, a standard approach is to break ties using original indices (stable tie-breaking): when $A[j] = A[i]$ and $j < i$, then $A[j]$ is treated as “smaller” than $A[i]$ for counting purposes. This guarantees a valid permutation of indices and produces a stable ordering relative to the input.

3.2 Time Complexity Analysis

- The algorithm conceptually performs comparisons over all ordered pairs (i, j) , which is $\Theta(n^2)$ comparisons.
- The final placement/copy steps are linear, $\Theta(n)$.

Hence, the running time is:

$$T(n) = \Theta(n^2).$$

The algorithm requires auxiliary arrays proportional to the input size, so:

$$S(n) = \Theta(n).$$

3.3 Testing Methodology (Conceptual)

To validate correctness, tests should include:

- arrays with many duplicates to confirm tie-handling,
- already-sorted and reverse-sorted arrays,
- random arrays, verifying (i) nondecreasing order and (ii) that the output is a permutation of the input.

4 Problem 3: DistributionCountingSort

4.1 Algorithm Description

DistributionCountingSort (Counting Sort) is a non-comparison sorting algorithm for integer keys in a bounded range. Rather than comparing pairs, it computes the frequency of each key and then uses cumulative counts to place each element directly into its correct position.

If negative values appear, the typical approach is to shift keys by subtracting the minimum value so that all keys map to indices starting at 0. Stability is commonly ensured by processing the input from right to left during placement.

4.2 Time Complexity Analysis

Let n be the number of elements and let $k = \max(A) - \min(A)$ be the width of the key range.

- Counting frequencies scans the input once: $\Theta(n)$.
- Building cumulative counts scans the range: $\Theta(k)$.
- Placement into the output scans the input once: $\Theta(n)$.

Therefore:

$$T(n, k) = \Theta(n + k).$$

Space usage includes an output array of size n and a count array of size $k + 1$:

$$S(n, k) = \Theta(n + k).$$

4.3 Comparison With ComparisonCountingSort

- **Complexity**
 - ComparisonCountingSort: $\Theta(n^2)$ independent of value range.
 - DistributionCountingSort: $\Theta(n + k)$, which can be significantly faster when k is small relative to n .
- **Assumptions**
 - ComparisonCountingSort works on any totally ordered type (numbers, strings with an order, etc.).
 - DistributionCountingSort requires integer-like keys that can be mapped into a small range.
- **Implementation differences**
 - ComparisonCountingSort relies on pairwise comparisons and careful duplicate handling via tie-breaking.
 - DistributionCountingSort relies on counting and prefix sums and requires additional memory proportional to k .

4.4 Testing Methodology (Conceptual)

To validate correctness and observe performance behavior, tests should include:

- arrays with negative and positive keys,
- arrays with small k (dense range) and large k (sparse range),
- duplicate-heavy arrays to verify stability (if required by the implementation).

5 Problem 4: Hashing With Closed Hashing (Open Addressing)

5.1 Algorithm Description

A hash table stores keys in an array of fixed size m . When multiple keys map to the same slot, a collision occurs. Under **closed hashing** (also known as **open addressing**), collisions are resolved by probing for another slot within the same table array.

A straightforward method is **linear probing**, where the probe sequence checks consecutive positions modulo m . Deletion typically uses a **tombstone marker** so that searches do not break probe chains: slots may be in one of three conceptual states (never used, currently occupied, or deleted).

5.2 Time Complexity Analysis

Let the load factor be $\alpha = \frac{n}{m}$.

- **Average case:** Under standard uniform hashing assumptions and moderate α , insert/search/delete operations are expected to run in $\Theta(1)$ time.
- **Worst case:** In the presence of clustering or when the table is nearly full, an operation may require probing many slots, up to $\Theta(m)$.

The space complexity is $\Theta(m)$ for the underlying table.

6 Complexity Summary

Table 1: Time Complexity of Algorithms

Algorithm	Time Complexity	Space Complexity
In-place swap (arith./XOR)	$\Theta(1)$	$\Theta(1)$
ComparisonCountingSort	$\Theta(n^2)$	$\Theta(n)$
DistributionCountingSort	$\Theta(n + k)$	$\Theta(n + k)$
Closed Hashing (linear probing)	Avg. $\Theta(1)$, Worst $\Theta(m)$	$\Theta(m)$

Repository

Full source code available at: [Aispur Santiago - Analysis of Algorithms, GitHub Repository](#)